JAVA CORE
SPRING FRAMEWORK
& SPRING BOOT

STUDY GUIDE & INTERVIEW PREPARATION

Đề cương học tập và ôn phỏng vấn theo mô hình 3 tầng

Core Internals → Production Engineering → Interview & System Reasoning

Đối tượng: Java Backend Intern/Junior; người đã hoặc đang xây dựng ứng dụng bằng Java và Spring Boot.

Phạm vi: Java language, JVM, Collections, Generics, Concurrency; Spring Framework Core, Spring Boot, MVC, JPA, Transaction, Security, Testing và Production Engineering.

Baseline: Java 21. Kiểm tra phiên bản Spring Framework/Spring Boot thực tế của dự án trước khi áp dụng API hoặc cấu hình.

Mục tiêu không phải học thuộc câu trả lời hay annotation. Mục tiêu là giải thích được cơ chế, kiểm chứng bằng code/test, debug được lỗi và bảo vệ được lựa chọn kỹ thuật.

Tài liệu này mở rộng dựa trên file tham khảo “Java Core & Spring Core — Study Guide” (16 trang): giữ lại mô hình 3 tầng, khung trả lời 5 bước, bài tập, ngân hàng câu hỏi và checklist; đồng thời bổ sung Spring Boot và các chủ đề vận hành backend.

MỤC LỤC

- Phần I. Phương pháp luyện trả lời theo khung năm bước
- Phần II. Cách sử dụng tài liệu và kế hoạch học
- Phần III. Java Core — Language, Object Model và Runtime
- Phần IV. Java Core — Production Engineering và Concurrency
- Phần V. Spring Framework — Core Internals
- Phần VI. Spring Boot — Application Engineering
- Phần VII. Web, Data Access và Security
- Phần VIII. Production Engineering, Reliability và Observability
- Phần IX. Testing, Debugging và bài tập tích hợp
- Phần X. Ngân hàng 100 câu hỏi phỏng vấn
- Phần XI. Checklist tự đánh giá và kế hoạch ôn tập

Phạm vi

Tài liệu chia kiến thức thành hai nền tảng liên kết: Java Core giải thích ngôn ngữ, runtime, memory và concurrency; Spring Framework/Boot giải thích container, proxy, web pipeline, persistence, security và cách vận hành ứng dụng. Spring Boot không thay thế Spring Framework; Boot cung cấp convention, auto-configuration, starter và cách đóng gói/khởi chạy thuận tiện hơn.

Một số chi tiết phụ thuộc JVM implementation, database engine, Spring version hoặc cấu hình. Tài liệu đánh dấu các điểm cần xác minh thay vì đưa ra quy tắc tuyệt đối.

PHẦN I — PHƯƠNG PHÁP LUYỆN TRẢ LỜI

1. Definition — Khái niệm là gì?

Nêu định nghĩa, mục đích và vấn đề được giải quyết. Phân biệt với khái niệm gần giống.

2. Mechanism — Hoạt động như thế nào?

Mô tả luồng thực thi và thành phần tham gia: language semantics, JVM, memory model, container, proxy, transaction manager hoặc request pipeline.

3. Example — Ví dụ thực tế hoặc code minh họa.

Tạo ví dụ nhỏ, dự đoán output hoặc viết test chứng minh hành vi. Nên có trường hợp đúng và trường hợp dễ gây hiểu nhầm.

4. Limitation — Giới hạn và failure modes.

Nêu edge case, giới hạn, lỗi phổ biến hoặc hành vi phụ thuộc phiên bản/cấu hình.

5. Trade-off — Khi nào nên sử dụng?

So sánh phương án thay thế về correctness, performance, complexity, testability và maintainability.

Mẫu ghi chú

Trường	Ghi chú của người học
Question / Concept	Ghi lại câu hỏi chính xác và nhóm kiến thức.
Definition	Định nghĩa trong 2–4 câu bằng ngôn ngữ của bản thân.
Mechanism	Các bước hoạt động, thành phần tham gia, sơ đồ nếu cần.
Example	Code, test, output dự đoán hoặc tình huống thực tế.
Limitation	Ít nhất một edge case hoặc failure mode.
Trade-off	So sánh với ít nhất một phương án thay thế.
Confidence	0 = chưa hiểu; 1 = nhận diện; 2 = giải thích; 3 = áp dụng/debug.
Review date	Ngày cần tự trả lời lại mà không xem ghi chú.

- Không tính là hoàn thành nếu chỉ nhớ định nghĩa hoặc annotation.
- Phân biệt điều được specification bảo đảm với hành vi phụ thuộc implementation, database, configuration hoặc version.
- Với concept trọng tâm, cần có ví dụ code/test để xác minh.
- Không cần nói đủ năm bước trong mọi câu trả lời phỏng vấn; đây là khung luyện tập.

PHẦN II — CÁCH SỬ DỤNG TÀI LIỆU VÀ KẾ HOẠCH HỌC

Tầng	Mục tiêu	Câu hỏi tự kiểm tra
Tầng 1 — Core Internals	Hiểu ngôn ngữ, runtime, container, lifecycle, proxy và persistence.	Vì sao nó hoạt động như vậy? Thành phần nào tham gia?
Tầng 2 — Production Engineering	Áp dụng dưới tải, concurrency, lỗi dependency, nhiều instance và giới hạn tài nguyên.	Khi có lỗi/tải cao/restart, giải pháp còn đúng không?
Tầng 3 — Interview & System Reasoning	Giải thích, dự đoán output, debug và bảo vệ lựa chọn.	Tôi có thể đưa ví dụ, limitation và trade-off không?

Thứ tự ưu tiên

Ưu tiên	Chủ đề	Mức cần đạt
P0	OOP, equals/hashCode, Collections, Generics, JVM, Concurrency	Giải thích cơ chế và dự đoán hành vi code.
P0	IoC/DI, Bean lifecycle, AOP/Proxy, Transaction	Giải thích Spring internals và các lỗi thường gặp.
P0	Spring MVC, REST, JPA/Hibernate, Security	Xây dựng và debug Backend API.
P1	Boot auto-configuration, configuration, testing, Actuator	Cấu hình, kiểm thử và vận hành ứng dụng.
P1	Async, Scheduling, Retry, Cache	Xử lý background task và dependency failure.
P2	Messaging, Outbox, Observability, Modulith	Hiểu reliability và khả năng mở rộng.

Nhịp học gợi ý

- Mỗi tuần chọn một nhóm lớn, chia thành các concept nhỏ.
- Mỗi ngày: đọc tài liệu chính thức, viết code tối thiểu và trả lời 5–10 câu không nhìn ghi chú.
- Cuối tuần: mock interview 30–60 phút; ghi lại câu thiếu cơ chế, ví dụ hoặc limitation.
- Mỗi 2 tuần: ôn lại các câu từng trả lời sai.
- Đánh giá tiến độ bằng khả năng giải thích, test và debug, không phải số trang đã đọc.

PHẦN III — JAVA CORE: LANGUAGE, OBJECT MODEL VÀ RUNTIME

Mục tiêu: hiểu ngữ nghĩa Java, dự đoán hành vi code và giải thích các cơ chế runtime nền tảng.

1. OOP, type system và object model

- Encapsulation, abstraction, inheritance, polymorphism; composition over inheritance.
- Interface, abstract class, default methods; dynamic dispatch, method overloading và overriding.
- Class, object, reference, object identity; Java pass-by-value và reference semantics.
- == và equals(); equals/hashCode contract: reflexive, symmetric, transitive, consistent và null handling.
- static, final, this, super; access modifiers và scope.
- Initialization order: superclass/subclass, static fields/blocks, instance fields/blocks và constructor.
- Nested, static nested, inner, local và anonymous classes.
- String immutability, string pool, StringBuilder/StringBuffer và object allocation.
- Records, sealed classes và pattern matching: mục đích, giới hạn và version.

Câu hỏi trọng tâm

- Java truyền tham số theo value hay reference? Thay đổi object và gán lại reference khác nhau thế nào?
- Vì sao override equals() thường phải override hashCode()?
- Overloading được quyết định lúc compile time hay runtime? Overriding thì sao?
- final reference có làm object immutable không?
- Vì sao String immutable và điều đó ảnh hưởng đến pooling, security, concurrency ra sao?

2. Generics

- Generic class, interface, method; type parameter và bounded type parameters.
- Type erasure: điều gì còn và không còn ở runtime; hạn chế generic array, instanceof và primitive type arguments.
- Raw types, unchecked warning, heap pollution.
- Wildcards: ?, ? extends T, ? super T; PECS — Producer Extends, Consumer Super.
- Invariance: vì sao List không phải subtype của List.
- Type inference, diamond operator, target typing và bridge methods ở mức khái niệm.

```
List<? extends Number> source = List.of(1, 2, 3);
Number value = source.get(0);
// source.add(4); // không h■p l■

List<? super Integer> target = new ArrayList<Number>();
target.add(4);
```

3. Collections Framework

Structure	Implementation	Điểm cần hiểu
List	ArrayList	Dynamic array, amortized append, random access.
List/Deque	LinkedList	Doubly linked nodes, traversal cost, memory overhead.
Set	HashSet	Uniqueness dựa trên equality và hash.
Set	TreeSet	Sorted order, comparator và balanced tree.
Map	HashMap	Hashing, bucket, collision, resize, load factor.
Map	LinkedHashMap	Insertion/access order.
Map	TreeMap	Sorted keys và comparator.
Queue/Deque	ArrayDeque	FIFO/LIFO, thao tác hai đầu.
Queue	PriorityQueue	Heap-based priority ordering.

- Collection hierarchy: List, Set, Queue, Deque; Map không kế thừa Collection.
- HashMap: hash spreading, bucket indexing, collision handling, resize và treeification theo implementation/version.
- equals/hashCode ảnh hưởng trực tiếp tới lookup và uniqueness.
- Comparable vs Comparator; comparator phải nhất quán với yêu cầu equality/order.
- Iterator, fail-fast behavior và ConcurrentModificationException; fail-fast không phải cơ chế đồng bộ.

- List.of/Map.of và Collections.unmodifiableX là unmodifiable view/value, không tự động làm object lồng bên trong immutable.
- Chọn collection theo complexity, ordering semantics, memory overhead và workload.

4. Exceptions và resource management

- Throwable hierarchy: Error, Exception, RuntimeException; checked vs unchecked.
- try/catch/finally, multi-catch, throw vs throws, exception chaining và cause.
- Try-with-resources, AutoCloseable và suppressed exceptions.
- Custom exception; giữ nguyên nguyên nhân khi wrap lỗi.
- Không dùng exception làm luồng điều khiển thông thường khi có lựa chọn rõ ràng hơn.
- Optional: map, flatMap, orElse, orElseGet, orElseThrow; tránh lạm dụng Optional trong field/parameter.

5. Functional Java và Stream API

- Functional interface, lambda, method reference và target typing.
- Predicate, Function, Consumer, Supplier và primitive specializations.
- Stream pipeline: source, intermediate operations, terminal operations và lazy evaluation.
- map, flatMap, filter, reduce, collect, groupingBy, partitioningBy.
- Stateless/stateful operations, encounter order và side effects.
- Sequential vs parallel streams; common ForkJoinPool và shared-state hazards.
- Stream vs loop: readability, debugging, allocation và performance.

```
orders.stream()
    .filter(order -> order.isPaid())
    .flatMap(order -> order.items().stream())
    .collect(Collectors.groupingBy(
        Item::category,
        Collectors.counting()
    ));
```

6. JVM và runtime

- JDK, JVM và Java runtime; source code → bytecode → class loading → interpretation/JIT execution.
- Class loading: loading, linking (verification, preparation, resolution) và initialization.
- ClassLoader delegation và class identity.
- Runtime data areas: heap, Java stacks, PC registers, native method stacks, method area/metaspace.
- Object reachability, reference types và garbage collection ở mức khái niệm.
- JIT compilation, warm-up, profiling; benchmark đơn giản dễ sai do warm-up, dead-code elimination và môi trường.
- GC goals: throughput, latency, pause time; đọc log/metrics ở mức cơ bản.
- Memory leak vẫn xảy ra khi object không còn cần nhưng vẫn reachable.
- Heap dump, thread dump, JFR và profiler.

7. Modern Java và API design

- Enum, annotations, var, records, sealed classes và pattern matching theo version.
- Date/Time API: Instant, LocalDate, LocalDateTime, ZonedDateTime, Duration, Period.
- BigDecimal và rounding modes; không dùng double cho phép tính tiền cần decimal exactness.
- API design: cohesion, coupling, invariants, defensive copying và immutability.
- Serialization basics, versioning và rủi ro deserialize dữ liệu không tin cậy.
- Backward compatibility, deprecation và lựa chọn API theo Java version.

PHẦN IV — JAVA CORE: PRODUCTION ENGINEERING VÀ CONCURRENCY

1. Complexity và collection selection

- Time/space complexity: average, amortized và worst-case.
- Capacity planning, resize, hash collision và allocation pressure.
- Chọn data structure theo access pattern, ordering, uniqueness và concurrent access.
- Immutable snapshots, defensive copying và ownership của mutable state.

2. Concurrency và Java Memory Model

- Process vs thread; platform thread, virtual thread, lifecycle và interruption.
- Race condition, atomicity, visibility, ordering và data race.
- Happens-before; monitor lock; volatile semantics.
- synchronized, ReentrantLock, tryLock và lock discipline.
- AtomicInteger/AtomicLong/AtomicReference, CAS và giới hạn atomic operation.
- Deadlock, livelock, starvation, contention và cách nhận diện.
- ConcurrentHashMap, BlockingQueue, CopyOnWriteArrayList; thread safety không làm mọi chuỗi thao tác thành atomic.
- ExecutorService: pool sizing, queue capacity, rejection policy, shutdown và cancellation.
- Future, CompletableFuture: composition, exception, timeout và cancellation.
- Virtual threads: phù hợp nhiều blocking I/O tasks; không làm CPU-bound work tự nhanh hơn và không xóa giới hạn database/remote resources.
- Thread safety trong một JVM không đồng nghĩa consistency giữa nhiều process/instance.

Concept	Bảo đảm chính	Không bảo đảm
volatile	Visibility và ordering theo JMM.	Không làm compound operation như i++ thành atomic.
synchronized	Mutual exclusion và happens-before qua unlock/lock cùng monitor.	Không phối hợp lock giữa nhiều JVM.
AtomicInteger	Atomic operation trên một biến; CAS-based semantics.	Không tự bảo vệ invariant liên quan nhiều biến.
ConcurrentHashMap	Thread-safe operations theo contract.	Không làm mọi workflow nhiều bước atomic.

3. I/O, network và resource management

- java.io và java.nio; stream vs channel, buffer và selector ở mức phù hợp.
- Buffering, encoding, charset và xử lý dữ liệu lớn.
- File/path API, try-with-resources và lỗi I/O.
- Socket/network timeout, retry boundary và đóng tài nguyên.
- Resource exhaustion: file descriptor, connection, thread và memory.

4. Testing, debugging và performance

- JUnit, unit/integration test, test doubles và boundary testing.
- Concurrency test với latch/barrier và repeated execution; tránh test phụ thuộc timing.
- Logging có context; phân biệt symptom với root cause.
- JFR, profiler, heap dump, thread dump và GC logs.
- JMH cho microbenchmark; tránh kết luận từ một lần System.nanoTime().
- Phân tích allocation, GC pressure, contention và latency distribution.

5. Secure and maintainable Java

- Input validation, output encoding và không tin cậy serialized input.
- Quản lý secrets; không ghi dữ liệu nhạy cảm vào log.
- Exception message không để lộ thông tin nội bộ không cần thiết.
- Dependency/version management và cập nhật thư viện.
- Code review: correctness, concurrency, resource cleanup, testability và complexity.

PHẦN V — SPRING FRAMEWORK: CORE INTERNALS

1. IoC, DI và container

- IoC chuyển quyền tạo/kết nối dependency từ application code sang container; DI là một cách hiện thực hóa IoC.
- BeanFactory là container cơ bản; ApplicationContext bổ sung event, resource, message source và tích hợp framework.
- BeanDefinition là metadata mô tả bean, scope, constructor arguments, properties và lifecycle.
- Component scanning, @Component/@Service/@Repository, @Configuration/@Bean và @Import là các nguồn đăng ký bean.
- Constructor injection, setter injection, field injection; ưu tiên constructor cho dependency bắt buộc.
- @Primary, @Qualifier, bean name, generic type và required/optional dependency resolution.
- ObjectProvider cho deferred lookup và optional dependency.
- Circular dependency; constructor cycle thường không thể khởi tạo hoàn chỉnh.
- Phân biệt Spring Framework features với Spring Boot conditional auto-configuration.

2. Bean lifecycle và scope

- Bean definition registration → instantiation → dependency population → aware callbacks → post-processing → initialization → ready → destruction.
- BeanFactoryPostProcessor tác động bean definitions; BeanPostProcessor tác động bean instances.
- @PostConstruct, InitializingBean, custom init method; @PreDestroy, DisposableBean, custom destroy method.
- Singleton là một instance trong container, không có nghĩa thread-safe.
- Prototype tạo instance khi resolve; container không quản lý đầy đủ destruction lifecycle của prototype.
- Request/session/application scope cần web-aware context.
- Prototype inject vào singleton thường chỉ được resolve một lần; dùng provider/proxy khi cần instance mới.
- FactoryBean và bean product là hai đối tượng khác nhau.
- Lazy initialization trì hoãn tạo bean và có thể trì hoãn phát hiện lỗi.

3. AOP và proxy

- Terminology: aspect, advice, pointcut, join point, target, proxy và weaving.
- Spring AOP chủ yếu proxy-based; invocation phải đi qua proxy để advice được áp dụng.
- JDK dynamic proxy dựa trên interface; CGLIB dựa trên subclass, có giới hạn final class/method.
- Advice: Before, After, AfterReturning, AfterThrowing, Around.
- Self-invocation không đi qua proxy bên ngoài; @Transactional/@Async có thể không hoạt động như kỳ vọng.
- Private/final method và proxy type tạo ra các giới hạn interception.
- Spring AOP vs AspectJ weaving; dùng đúng mức độ phức tạp cần thiết.

4. Transaction management

- Declarative transaction và TransactionInterceptor; PlatformTransactionManager điều phối transaction.
- @Transactional tạo boundary khi invocation đi qua proxy phù hợp.
- Propagation: REQUIRED, REQUIRES_NEW, SUPPORTS, NOT_SUPPORTED, MANDATORY, NEVER, NESTED.
- Isolation: READ_UNCOMMITTED, READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE; semantics cụ thể phụ thuộc database.
- Rollback mặc định thường áp dụng unchecked exception và Error; checked exception cần rollback rule nếu muốn rollback.
- readOnly là hint/setting, không phải authorization hay bảo đảm mọi write bị chặn.
- REQUIRES_NEW suspend transaction ngoài và có thể cần connection bổ sung.
- NESTED phụ thuộc transaction manager/resource support; không mặc định tương đương REQUIRES_NEW.
- Không kéo dài transaction qua network call chậm nếu không cần thiết.
- Self-invocation, private/final method và transaction manager selection là các điểm cần debug.

5. Events, resources và expression

- ApplicationEventPublisher, ApplicationListener, @EventListener; synchronous/asynchronous event.
- Application event nội bộ không phải durable message broker.
- @TransactionalEventListener có thể gắn listener với BEFORE_COMMIT/AFTER_COMMIT/AFTER_ROLLBACK/AFTER_COMPLETION.
- Resource abstraction hỗ trợ classpath, filesystem và URL.

- Environment, PropertySource, profiles và property resolution.
- SpEL, Type Conversion, Formatter/Converter và MessageSource ở mức cần thiết.
- ApplicationContext startup/shutdown hooks; tránh giấu business workflow trong lifecycle hook.

PHẦN VI — SPRING BOOT: APPLICATION ENGINEERING

1. Startup và auto-configuration

- @SpringBootApplication kết hợp @SpringBootConfiguration, @EnableAutoConfiguration và @ComponentScan.
- SpringApplication điều phối startup, Environment preparation, ApplicationContext creation/refresh và runners.
- Auto-configuration dựa trên classpath, properties, existing beans và conditional annotations.
- @ConditionalOnClass, @ConditionalOnMissingBean, @ConditionalOnProperty và các condition khác.
- Back-off: cấu hình mặc định nhường chỗ khi ứng dụng cung cấp bean tùy chỉnh phù hợp.
- Starter gom dependency thường dùng; embedded server hỗ trợ executable application.
- Condition Evaluation Report và failure analysis hỗ trợ debug startup.

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

2. Externalized configuration

- application.properties/YAML, environment variables, command-line arguments và các PropertySource.
- Property precedence phụ thuộc nguồn và phiên bản; kiểm tra đúng Spring Boot reference.
- @Value phù hợp property đơn lẻ; @ConfigurationProperties phù hợp nhóm cấu hình có cấu trúc và validation.
- Profiles tách cấu hình theo môi trường; không dùng profile thay cho secret management.
- Secret nên được inject từ môi trường/secret store; không commit vào repository.
- Configuration cần validation và fail-fast với giá trị bắt buộc.

```
@ConfigurationProperties(prefix = "reservation")
@Validated
public record ReservationProperties(
    @Min(1) int holdMinutes,
    @NotBlank String currency
) {}
```

3. Logging, Actuator và deployment

- Actuator cung cấp health, metrics, info và operational endpoints; chỉ expose endpoint cần thiết.
- Liveness trả lời process có cần restart không; readiness trả lời instance có nên nhận traffic không.
- Micrometer là instrumentation facade; backend có thể là Prometheus hoặc hệ thống khác.
- Structured logs nên có timestamp, level, trace/correlation ID và context không nhạy cảm.
- Executable JAR, embedded server, container image, JVM/container memory và graceful shutdown.
- Không expose env/config/heapdump hoặc endpoint nhạy cảm ra public.

4. Spring Boot version awareness

Tài liệu lấy Java 21 làm baseline. Spring Boot 3.x thuộc thế hệ Spring Framework 6.x; Spring Boot 4.x thuộc thế hệ mới hơn. Dependency, package, test annotation và configuration có thể thay đổi. Khi triển khai, hãy kiểm tra compatibility matrix và documentation của đúng version; không copy cấu hình từ bài viết cũ một cách máy móc.

PHẦN VII — WEB, DATA ACCESS VÀ SECURITY

1. Spring MVC request lifecycle

```
HTTP Request
   -> Servlet Filters
   -> DispatcherServlet
   -> HandlerMapping
   -> HandlerAdapter
   -> Controller
   -> Service / Use Case
   -> Repository
   -> Database
   -> HttpMessageConverter
   -> HTTP Response
```

- DispatcherServlet là Front Controller; HandlerMapping tìm handler và HandlerAdapter thực thi handler.
- Argument resolvers xử lý @PathVariable, @RequestParam, @RequestBody.
- HttpMessageConverter chuyển đổi body; Jackson thường xử lý JSON.
- Filter chạy ở Servlet container chain; HandlerInterceptor chạy trong Spring MVC lifecycle.
- Controller nên tập trung HTTP mapping và boundary; business rule nằm ở application/domain layer.
- @RestController, ResponseEntity, @ControllerAdvice và @ExceptionHandler.

2. REST API, DTO và validation

- Resource-oriented URI, HTTP method semantics, status code, idempotency và backward compatibility.
- DTO tách API contract khỏi persistence entity; tránh expose entity/lazy proxy.
- @Valid/@Validated kiểm tra request boundary; domain invariant vẫn phải được bảo vệ trong business layer.
- Pagination/sorting cần giới hạn page size và whitelist sort fields.
- Error response có code ổn định, message phù hợp và correlation ID; không lộ stack trace.
- OpenAPI mô tả contract nhưng không thay thế integration test.

3. Exception handling

- @RestControllerAdvice chuẩn hóa lỗi API; @ExceptionHandler xử lý exception.
- Phân loại validation, not found, conflict, unauthorized/forbidden và unexpected error.
- Giữ cause khi wrap; không swallow exception hoặc trả 200 cho lỗi nghiệp vụ.
- Log unexpected failure ở boundary có đủ context, tránh log trùng nhiều tầng.

4. Spring Data JPA và Hibernate

- JPA là specification; Hibernate là implementation; Spring Data JPA cung cấp repository abstraction.
- Entity lifecycle: transient, managed, detached, removed.
- Persistence Context quản lý managed entity; dirty checking phát hiện thay đổi.
- save() không đồng nghĩa SQL INSERT ngay lập tức; flush đồng bộ persistence context xuống DB nhưng không commit.
- First-level cache gắn với persistence context; second-level cache là cơ chế khác.
- Lazy loading có thể gây query ngoài dự kiến hoặc LazyInitializationException.
- N+1: một query lấy danh sách, sau đó mỗi item phát sinh query; cân nhắc fetch join, entity graph, batch fetching hoặc projection.
- Cascade lan truyền operation; orphanRemoval xử lý child bị loại khỏi association theo mapping.
- Không dùng EAGER như cách chữa N+1 một cách máy móc.
- Optimistic locking với @Version; pessimistic locking dùng database lock và có thể gây contention/deadlock.

Concept	Ý nghĩa	Failure mode
Persistence Context	Identity map và dirty checking.	Hiểu nhầm save() luôn phát SQL.
Flush vs Commit	Đồng bộ SQL vs kết thúc transaction.	Flush xong vẫn có thể rollback.
Lazy/Eager	Chiến lược tải quan hệ.	N+1 hoặc LazyInitializationException.
Cascade	Lan truyền thao tác theo association.	Xóa/ghi ngoài ý muốn.
@Version	Phát hiện concurrent update.	OptimisticLockException.
Pessimistic lock	Khóa dữ liệu ở database.	Lock wait, deadlock, giảm throughput.

5. Spring Security

- SecurityFilterChain định nghĩa chuỗi filter và request matching rules.
- Authentication xác minh danh tính; Authorization quyết định quyền.
- AuthenticationManager điều phối AuthenticationProvider; UserDetailsService tải user trong username/password flow.
- SecurityContext chứa Authentication; SecurityContextHolder cung cấp access theo strategy.
- PasswordEncoder hash password; không lưu plaintext hoặc tự thiết kế password hashing.
- Role/authority cần semantics rõ; method security có thể bổ sung authorization.
- JWT bearer flow cần validate signature, issuer/audience, expiry, algorithm và key rotation.
- Stateless không tự giải quyết refresh, logout/revocation hoặc token theft.
- CORS là browser cross-origin policy; CSRF là kiểu tấn công lợi dụng credential tự động của browser.

PHẦN VIII — PRODUCTION ENGINEERING, RELIABILITY VÀ OBSERVABILITY

1. Async và Executor

- @Async hoạt động qua proxy/interceptor và executor; self-invocation có thể bypass.
- Cấu hình core/max pool, queue capacity, thread naming và rejection policy theo workload.
- Queue không giới hạn có thể che giấu overload và làm memory tăng.
- Xử lý exception khác nhau giữa void async method và Future/CompletableFuture.
- Context propagation cho MDC, security context và tracing cần được thiết kế khi đổi thread.
- Async trong một process không phải durable job queue; restart có thể làm mất task chờ.

2. Scheduling và distributed jobs

- @Scheduled hỗ trợ fixedRate, fixedDelay, cron, initialDelay và timezone.
- Trong nhiều application instance, mỗi instance có thể chạy cùng scheduled method.
- Task dài có thể overlap tùy scheduler/configuration; kiểm soát concurrency.
- Distributed lock, database claim/lease hoặc Quartz có thể cần thiết tùy yêu cầu.
- Job nên idempotent, có checkpoint và retry budget.
- Chọn @Scheduled cho task đơn giản; Quartz cho persistent/complex scheduling; broker/worker cho queue, retry và scale consumer.

3. Retry, timeout, circuit breaker và bulkhead

Cơ chế	Giải quyết	Không giải quyết
Timeout	Giới hạn thời gian chờ dependency.	Không hủy side effect remote đã nhận.
Retry	Thử lại lỗi tạm thời.	Không làm thao tác non-idempotent an toàn.
Backoff + jitter	Giảm retry đồng loạt.	Không thay thế capacity planning.
Circuit breaker	Ngừng gọi dependency đang lỗi.	Không tự sửa dependency.
Bulkhead	Cô lập tài nguyên giữa workload.	Không loại bỏ mọi overload.
Rate limiter	Giới hạn tốc độ request.	Không tự bảo đảm fairness.

- Không retry validation/business rejection một cách mù quáng.
- Đặt timeout tổng thể và từng hop; retry budget phải nằm trong deadline.
- Tránh retry ở nhiều layer cùng lúc làm nhân số lần gọi.
- Retry thao tác ghi cần idempotency hoặc deduplication.

4. Cache và Redis

- Spring Cache là abstraction; Redis là implementation với network, serialization, TTL và failure modes.
- @Cacheable, @CachePut, @CacheEvict cần key strategy và invalidation policy.
- Cache-aside phù hợp read-heavy workload; xác định invalidation sau write.
- Cache stampede có thể giảm bằng single-flight, lock ngắn, jittered TTL hoặc stale-while-revalidate.
- Cache không phải source of truth cho reservation/inventory nếu consistency yêu cầu database làm nguồn chuẩn.

5. Events, Kafka/RabbitMQ và Outbox

- Application event phù hợp giao tiếp nội bộ; không có durability mặc định như broker.
- Broker hỗ trợ giao tiếp async giữa process; cần hiểu acknowledgement, delivery, retry và dead-letter.
- At-least-once có thể tạo duplicate; consumer cần idempotency/deduplication.
- Outbox ghi business change và event record trong cùng database transaction; publisher chuyển outbox record sang broker.
- Outbox giảm failure window giữa DB commit và publish; vẫn cần xử lý duplicate publish và consumer idempotency.

```
Business Transaction:
  update business tables
  insert outbox event
  COMMIT

Outbox Publisher:
  read unpublished events
  publish to broker
```

```
  mark published / retry

Consumer:
  validate / deduplicate
  apply idempotent state change
```

6. Observability và graceful shutdown

- Logs: sự kiện; metrics: xu hướng/số liệu; traces: request đi qua service/hop nào.
- Theo dõi latency p50/p95/p99, error rate, throughput, saturation và dependency health.
- Correlation ID/trace ID nên đi xuyên request và background processing.
- Graceful shutdown: ngừng nhận traffic, chờ in-flight work trong giới hạn, đóng executor/pool/connection.
- Alert nên gắn với user impact hoặc SLO, tránh cảnh báo nhiễu.

PHẦN IX — TESTING, DEBUGGING VÀ BÀI TẬP TÍCH HỢP

1. Testing strategy

Loại test	Phạm vi	Khi dùng
Unit test	Logic/class thuần Java.	Business rule, validation, mapping.
Slice test	Một phần Spring context.	MVC mapping, serialization, JPA slice.
Integration test	Nhiều thành phần thật trong context.	DI, transaction, persistence, security.
End-to-end	Ứng dụng qua interface gần production.	Critical user journey và contract.

- @SpringBootTest khởi tạo context lớn; mạnh nhưng tốn thời gian.
- @WebMvcTest tập trung MVC; @DataJpaTest tập trung JPA.
- Mockito @Mock tạo mock; @InjectMocks tạo object và inject mocks theo quy tắc Mockito, không khởi tạo Spring container.
- Testcontainers giúp test với database/broker thực tế.
- Test async/retry bằng latch, controlled clock hoặc Awaitility thay vì sleep dài.
- Test transaction cần kiểm tra dữ liệu sau commit/rollback đúng boundary.

2. Debugging checklist

Triệu chứng	Hướng điều tra
No qualifying bean	Component scan, profile, conditional config, import.
NoUniqueBeanDefinition	@Primary, @Qualifier, generic type, bean name.
Circular dependency	Dependency graph; chuyển về constructor injection rõ ràng.
@Transactional không chạy	Proxy, self-invocation, visibility, manager, rollback rule.
LazyInitializationException	Persistence context boundary, fetch plan, DTO mapping.
N+1 query	SQL log, statistics, fetch join/entity graph/projection.
Connection pool exhausted	Transaction duration, slow query, leaked connection, pool sizing.
Async không chạy	Enable async, proxy call, executor, rejection, exception.
Scheduled job chạy trùng	Số instance, distributed coordination, idempotency.

3. Bài tập Java Core

- Viết test equals/hashCode và dùng object làm HashMap key; thay đổi field tham gia hash để quan sát lỗi.
- So sánh ArrayList, LinkedList, HashSet, TreeSet với workload khác nhau.
- Tạo ví dụ generics extends/super và giải thích compiler rejection.
- Viết Stream pipeline có groupingBy/flatMap; so sánh với loop.
- Tạo race condition counter; sửa bằng synchronized và AtomicInteger.
- Dùng ExecutorService xử lý batch với giới hạn thread, exception handling và shutdown.
- Tạo CompletableFuture composition có timeout và recovery.
- Tạo memory leak có chủ đích bằng collection giữ reference; quan sát heap dump.

4. Bài tập Spring Framework/Boot

- Tạo hai implementation cùng interface; tái hiện ambiguity rồi sửa bằng @Primary/@Qualifier.
- Viết BeanPostProcessor ghi lại lifecycle; thử singleton phụ thuộc prototype.
- Viết logging aspect; so sánh external invocation với self-invocation.
- Tạo transaction REQUIRED/REQUIRES_NEW và integration test rollback.
- Xây dựng REST endpoint có DTO, validation, exception contract và MockMvc test.
- Tái hiện N+1 rồi sửa bằng fetch join/projection; so sánh SQL count.
- Test optimistic locking khi hai request cập nhật cùng record.
- Cấu hình SecurityFilterChain và test unauthenticated/forbidden/authorized.
- Tạo scheduled job idempotent và thử chạy trên nhiều instance.
- Xây outbox publisher/consumer; test crash sau publish nhưng trước mark-as-published.

5. Bài tập gắn với RushSeat

Ưu tiên correctness trước khi thêm công nghệ. Với flow giữ chỗ, cần xác định transaction boundary, invariant trạng thái, locking strategy và hành vi khi request đồng thời.

- Trong một transaction phù hợp: kiểm tra event đang mở bán, kiểm tra inventory và tạo reservation hold.
- Thử hai request đồng thời giữ cùng một seat/ticket inventory; xác minh không oversell.
- Thiết kế expiration job có thể chạy lại; transition chỉ áp dụng nếu reservation còn HELD và đã hết hạn.
- Kiểm tra duplicate confirmation/callback; dùng idempotency key hoặc state transition guard.
- Đưa event vào outbox cùng transaction tạo reservation; consumer cập nhật side effect một cách idempotent.
- Đo p95 latency, throughput, DB connection usage và lock contention trước/sau tối ưu.

PHẦN X — NGÂN HÀNG 100 CÂU HỎI PHỎNG VẤN

Các câu hỏi chỉ gồm đề bài. Tự trả lời theo khung năm bước; ưu tiên tạo code/test cho câu hỏi về cơ chế.

Java Core A — OOP, object model và language

001. Java truyền tham số theo value hay reference? Giải thích bằng ví dụ object.
002. Phân biệt ==, equals() và hashCode(); hợp đồng giữa equals/hashCode là gì?
003. Vì sao String immutable? Điều đó ảnh hưởng pooling, security và concurrency ra sao?
004. Overloading khác overriding thế nào? Method được chọn ở compile time và runtime ra sao?
005. Interface khác abstract class ở đâu? Khi nào composition phù hợp hơn inheritance?
006. Trình bày thứ tự khởi tạo static/instance field, block và constructor giữa superclass/subclass.
007. final áp dụng cho variable, method và class có ý nghĩa gì?
008. Checked exception và unchecked exception khác nhau thế nào?
009. Optional nên được dùng ở đâu và khi nào làm API khó hiểu hơn?
010. Record khác class thông thường thế nào? Record có làm nested object immutable không?

Java Core B — Generics, Collections và Functional

011. Type erasure là gì và tạo ra giới hạn nào?
012. Giải thích PECS và phân biệt ? extends T với ? super T.
013. Vì sao List không phải subtype của List?
014. HashMap tìm key như thế nào? Vai trò hashCode, equals, collision và resize?
015. Điều gì xảy ra nếu object làm key trong HashMap bị thay đổi field tham gia hashCode?
016. ArrayList và LinkedList khác nhau thế nào về complexity và workload?
017. HashSet và TreeSet khác nhau thế nào về uniqueness, ordering và cost?
018. Comparable khác Comparator thế nào?
019. Stream lazy evaluation là gì? Intermediate và terminal operations khác nhau ra sao?
020. Parallel stream có thể chậm hơn sequential stream trong trường hợp nào?

Java Core C — JVM, memory và concurrency

021. Phân biệt JDK, JVM và Java runtime; source code được thực thi như thế nào?
022. Class loading, linking và initialization là gì?
023. Heap, Java stack và metaspace chứa loại dữ liệu nào ở mức khái niệm?
024. GC xác định object có thể thu hồi như thế nào? Vì sao không dựa vào GC để đóng resource?
025. Memory leak trong Java có thể xảy ra dù có GC không?
026. Phân biệt atomicity, visibility và ordering.
027. volatile bảo đảm điều gì và không bảo đảm điều gì?
028. synchronized liên hệ với happens-before như thế nào?
029. Race condition, deadlock, livelock và starvation khác nhau thế nào?
030. AtomicInteger/CAS khác synchronized ở đâu?

Java Core D — Production và API

031. ExecutorService giải quyết vấn đề gì so với tự tạo Thread cho mỗi task?
032. Future khác CompletableFuture thế nào?
033. Platform thread khác virtual thread ở đâu?
034. ConcurrentHashMap khác HashMap thế nào? Thread-safe collection có làm workflow nhiều bước atomic không?
035. Try-with-resources và suppressed exception hoạt động thế nào?
036. Instant, LocalDateTime và ZonedDateTime khác nhau thế nào?
037. Vì sao dùng BigDecimal cho tiền và cần chú ý rounding/scale?
038. Benchmark bằng System.nanoTime một lần có thể gây hiểu lầm thế nào?
039. Khi CPU hoặc memory tăng bất thường, bạn điều tra theo thứ tự nào?
040. Thiết kế immutable API cần quan tâm constructor, defensive copying và nested mutable objects ra sao?

Spring Core E — IoC, DI và lifecycle

041. IoC khác DI thế nào? Spring container giải quyết vấn đề gì?
042. BeanFactory và ApplicationContext khác nhau ở đâu?
043. BeanDefinition là gì? Spring đăng ký bean từ những nguồn nào?
044. @Component khác @Bean thế nào?

045. @Primary và @Qualifier giải quyết vấn đề gì?
046. Trình bày các giai đoạn chính của bean lifecycle.
047. BeanFactoryPostProcessor khác BeanPostProcessor thế nào?
048. Singleton scope có đồng nghĩa thread-safe không?
049. Circular dependency là gì? Vì sao constructor cycle thường không khởi tạo được?
050. @Lazy ảnh hưởng thời điểm khởi tạo và phát hiện lỗi ra sao?

Spring Core F — AOP, Proxy và Transaction

051. AOP giải quyết vấn đề nào? Giải thích aspect, advice, pointcut, join point và target.
052. JDK Dynamic Proxy khác CGLIB proxy thế nào?
053. Vì sao self-invocation có thể làm @Transactional hoặc @Async không hoạt động?
054. @Around advice có thể thay đổi kết quả method như thế nào?
055. Declarative transaction hoạt động qua những thành phần nào?
056. REQUIRED khác REQUIRES_NEW thế nào?
057. Những exception nào mặc định khiến transaction rollback?
058. Isolation level và propagation khác nhau thế nào?
059. Vì sao gọi HTTP API chậm bên trong transaction có thể gây vấn đề?
060. Spring AOP proxy khác AspectJ weaving thế nào?

Spring Boot G — Startup và configuration

061. @SpringBootApplication bao gồm những annotation nào?
062. Auto-configuration dựa trên những điều kiện nào?
063. @ConditionalOnClass và @ConditionalOnMissingBean khác nhau thế nào?
064. Auto-configuration back-off là gì?
065. Starter dependency giải quyết vấn đề gì?
066. Embedded server thay đổi cách deploy ứng dụng như thế nào?
067. @Value và @ConfigurationProperties khác nhau thế nào?
068. Property source precedence ảnh hưởng runtime ra sao?
069. Profile có nên được dùng để lưu secret không?
070. Debug missing/ambiguous bean khi startup thất bại theo quy trình nào?

Spring Web H — MVC và REST

071. DispatcherServlet đóng vai trò gì?
072. Request đi qua HandlerMapping và HandlerAdapter như thế nào?
073. Filter khác HandlerInterceptor ở đâu?
074. @Controller khác @RestController thế nào?
075. @RequestBody được chuyển thành object bằng cơ chế nào?
076. HttpMessageConverter và Jackson có vai trò gì?
077. DTO khác Entity thế nào?
078. @Valid và @Validated được sử dụng ra sao?
079. @RestControllerAdvice giải quyết vấn đề gì?
080. Thiết kế error response ổn định cho REST API cần lưu ý gì?

Spring Data/Security I — Persistence và Security

081. JPA, Hibernate và Spring Data JPA khác nhau thế nào?
082. Persistence Context là gì?
083. Entity có những lifecycle state nào?
084. Dirty checking hoạt động như thế nào?
085. save(), flush() và commit() khác nhau thế nào?
086. Lazy loading và eager loading có trade-off gì?
087. N+1 query problem xảy ra thế nào và xử lý ra sao?
088. Cascade khác orphanRemoval thế nào?
089. Optimistic locking với @Version hoạt động ra sao?
090. Authentication khác Authorization thế nào?

Spring Production J — Async, reliability và testing

091. SecurityFilterChain có vai trò gì?
092. JWT validation cần kiểm tra những thuộc tính nào?

093. CORS khác CSRF thế nào?
094. @Async sử dụng cơ chế nào? Làm sao cấu hình executor?
095. fixedRate khác fixedDelay thế nào?
096. Vì sao @Scheduled có thể chạy cùng một job trên nhiều instance?
097. Retry nên áp dụng cho loại lỗi nào? Vì sao retry có thể tạo duplicate side effect?
098. Circuit breaker khác retry thế nào?
099. Outbox pattern giải quyết failure window nào?
100. @SpringBootTest khác test slice và Testcontainers như thế nào?

PHẦN XI — CHECKLIST TỰ ĐÁNH GIÁ

Java Core

- Tôi giải thích được OOP, pass-by-value, equals/hashCode và object lifecycle.
- Tôi hiểu generics, wildcards, PECS và type erasure.
- Tôi chọn được collection theo contract, complexity và workload.
- Tôi giải thích được class loading, heap/stack, GC và memory leak.
- Tôi phân biệt atomicity, visibility, ordering và happens-before.
- Tôi có thể viết/test/debug concurrency cơ bản.
- Tôi biết quản lý exception, I/O và resource đúng cách.

Spring Framework

- Tôi giải thích được IoC, DI, BeanFactory, ApplicationContext và BeanDefinition.
- Tôi mô tả được bean lifecycle, post-processors và scope.
- Tôi hiểu proxy, AOP, self-invocation và giới hạn proxy.
- Tôi giải thích được transaction boundary, propagation, isolation và rollback.
- Tôi hiểu application events và giới hạn durability.

Spring Boot và Backend

- Tôi giải thích được startup flow, auto-configuration và conditional registration.
- Tôi quản lý externalized configuration, profiles và secrets.
- Tôi mô tả được DispatcherServlet request lifecycle.
- Tôi hiểu JPA persistence context, dirty checking, flush/commit và N+1.
- Tôi hiểu SecurityFilterChain, Authentication, Authorization và JWT.
- Tôi cấu hình/test được async, scheduling, retry và cache ở mức phù hợp.
- Tôi biết sử dụng Testcontainers, Actuator, metrics và logs để debug.

Interview readiness

- Tôi trả lời được 5–10 câu mỗi ngày mà không nhìn tài liệu.
- Tôi có code/test minh họa cho concept trọng tâm.
- Tôi nêu được limitation, failure mode và trade-off.
- Tôi có thể debug một lỗi Spring Boot dựa trên log và test tái hiện.
- Tôi kiểm tra đúng Java/Spring version trước khi khẳng định behavior.

Rubric chấm câu trả lời

Điểm	Mô tả
0	Không biết hoặc trả lời sai bản chất.
1	Nhớ định nghĩa nhưng không giải thích được cơ chế.
2	Giải thích đúng cơ chế nhưng thiếu ví dụ hoặc limitation.
3	Có cơ chế, ví dụ chính xác và ít nhất một giới hạn.
4	So sánh trade-off, nêu giả định và áp dụng vào tình huống production.

Kế hoạch ôn tập cá nhân

Chủ đề	Điểm 0–4	Lỗ hổng cần sửa	Ngày ôn lại
Java — OOP/Object model			
Java — Generics/Collections			
Java — JVM/Memory			
Java — Concurrency			
Spring — IoC/Lifecycle			
Spring — AOP/Proxy			
Spring — Transactions			
Spring Boot — Auto Configuration			
Spring MVC/REST			
Spring Data JPA			
Spring Security			
Production/Testing/Observability			

Tài liệu tham khảo chính thức

- Java learning: https://dev.java/learn/
- Java Language Specification: https://docs.oracle.com/javase/specs/
- Java API: https://docs.oracle.com/en/java/javase/21/docs/api/
- Spring Framework Reference: https://docs.spring.io/spring-framework/reference/
- Spring Boot Reference: https://docs.spring.io/spring-boot/reference/
- Spring Data JPA Reference: https://docs.spring.io/spring-data/jpa/reference/
- Spring Security Reference: https://docs.spring.io/spring-security/reference/
- Testcontainers: https://testcontainers.com/
- Micrometer: https://docs.micrometer.io/

Cấu trúc và phương pháp được tham khảo từ file “Java Core & Spring Core — Study Guide” người dùng cung cấp (16 trang). Nội dung trong bản này được tổ chức lại thành một tài liệu thống nhất gồm Java Core, Spring Framework Core và Spring Boot; các chủ đề production được bổ sung để kết nối với công việc Backend thực tế.

Kết thúc tài liệu. Hãy ưu tiên hiểu sâu các concept P0, kiểm chứng bằng code/test và áp dụng vào project đang xây dựng.